

# Entangled Enhancement in Modular Autonomous Driving Systems: An Empirical Study of Cascade Effects from Single-Component Retraining

Rashedul Arefin Ifty, Fumio Machida

University of Tsukuba

Tsukuba, Japan

ifty.rashedul@sd.cs.tsukuba.ac.jp, machida@cs.tsukuba.ac.jp

**Abstract**—Machine learning systems in autonomous driving are maintained incrementally: one component is retrained as its domain drifts while the rest of the pipeline stays fixed. We study the downstream effect of such an update in a modular perception–prediction–planning pipeline, built from a YOLOv11n detector (C1), a constant-velocity predictor (C2), and a rule-based Intelligent Driver Model planner (C3). We train C1 on the full Boston nuScenes trainval split and fine-tune it on Singapore, measuring every component in two modes, *isolated* with ground-truth inputs and *pipeline* with live upstream inputs, over an evaluation set of 2,020 frames. Rather than a single retraining, we run a campaign of 15 updates varying data size, training length, and seed. The retrained detector improves on its own metric in every update, yet in 6 of the 15 the planner’s trajectory-agreement metric worsens at the same time. This is the *strict* form of the analytically modeled *entangled enhancement*. Crucially, the terminal metrics *diverge*: on the reference update the improved detector moved the planner’s L2 agreement worse (+0.124 m) while its collision rate improved, from 2% under the incumbent to zero. An upstream component better on its own and its neighbor’s metric thus pushes the terminal readings in opposite directions, so no single terminal metric admits it safely. The terminal effect is a landscape rather than a constant. Every isolated-mode metric remained bit-identical, confirming that all changes propagated through the component interfaces. Building on these findings we propose a cascade-aware retraining assessment method and evaluate its low-cost interface-drift front-end on a held-out split; the front-end tracks the magnitude of downstream disturbance but not its direction, positioning it as a prioritization filter rather than a standalone admission rule.

**Index Terms**—machine learning systems, autonomous driving, retraining, cascade effects, software reliability, modular pipelines, regression testing

## I. INTRODUCTION

Autonomous driving systems are built either as a single end-to-end (E2E) network that maps sensor input directly to a driving plan [1], [2], or as a modular pipeline of separately trained perception, prediction, and planning components. In deployment, such systems are maintained under domain drift one component at a time: a perception model is retrained for a new city, a predictor is updated as agent statistics shift, a planner is revised for new regulations. This component-scoped maintenance is only well-posed in a modular pipeline, where components are separable and independently retrainable. In an E2E model the smallest retrainable unit is the entire network, so we adopt the modular pipeline as the setting for this study.

Retraining one component while the rest stay fixed is the common case in practice, and it is also where a subtle hazard arises. This work builds directly on Wang and Machida [3], who model a two-component MLS as a continuous-time Markov chain and name the failure mode we study, *entangled enhancement*: improving an upstream component on its own metric can still degrade the system, because downstream components have learned to operate on the previous upstream output. A follow-on study casts this as an availability–continuity trade-off [4]. Both are stochastic models that need empirical calibration. So far no one has measured how a single-component retraining event actually propagates through a real perception–prediction–planning pipeline, nor how that propagation varies from one retraining to the next. That is the gap this paper addresses.

The missing element is a controlled measurement of this propagation on a real pipeline, and this paper supplies it. We build a three-component pipeline on the nuScenes dataset [5]: C1, a YOLOv11n [6] camera detector, C2, a constant-velocity trajectory predictor, and C3, a rule-based planner from the Intelligent Driver Model (IDM) family [7] whose proposal-and-score design follows PDM-Closed [8]. We train C1 on Boston scenes and fine-tune it on Singapore scenes, a well-known domain gap within nuScenes, keeping C2 and C3 frozen. The experiments center on two components. C1 is the retrained unit, and C2 is its direct consumer. The interface between them, the stream of detected objects, is where the cascade travels. C3 is kept as the system-level readout that reports whether the change helps or harms the final plan.

The main instrument is dual-mode evaluation, and it exists to separate a component’s own error from the error it inherits from upstream. Each downstream component is measured twice: in *isolated* mode with ground-truth inputs, which exposes only its own error, and in *pipeline* mode with live upstream outputs, which exposes its own error plus whatever it inherits. The difference between the two modes is the inherited part, so its change across a retraining event measures the cascade that the retraining injects.

A single retraining, however, is one point. To learn whether the cascade is a fixed property or a variable one, we run a *campaign*: 15 retraining updates of C1 that vary the amount of fine-tuning data, the number of training epochs, and the

random seed, each measured through the same dual-mode harness. This extends the study from a single observation to a landscape of retraining outcomes, and it motivates a practical follow-up question: can a low-cost check, computed from the detector’s outputs alone, predict which updates disturb the plan the most?

Our measurements yield six findings, detailed in Section V.

- F1. (Architectural.)** The component-level retraining question is unobservable in an E2E architecture and unaffordable at E2E training costs; a modular pipeline is the enabling architecture.
- F2. (Isolation.)** The isolation property that stochastic MLS models assume holds exactly: the isolated-mode metrics of the frozen  $C_2$  and  $C_3$  stayed bit-identical across every retraining event.
- F3. (Cascade existence.)** Retraining  $C_1$  alone produced large, measurable changes downstream, concentrated at the first  $C_1 \rightarrow C_2$  interface.
- F4. (Strict entangled enhancement, directly observed.)** On full trainval, retraining  $C_1$  improves its own metric ( $\delta_1 > 0$ ) in all 15 updates and improves  $C_2$  in all 15, yet in 6 of them the planner *worsens* at the same time. This is the strict condition (2), where the detector improves while the system degrades, observed directly in a real driving pipeline, with no class re-balancing required.
- F5. (A landscape, not a constant.)** Across the 15 updates the terminal effect varies: the planner is worse ( $\Delta_3 > 0$ ) in 6 and better in the other 9, with plan shift in 0.78–1.50 m. Which updates enter the entangled-enhancement regime is not predictable from the training configuration.
- F6. (The terminal metrics disagree.)** On 2,020 frames the incumbent pipeline has a non-zero collision rate (2%). Under retraining the two terminal metrics move in opposite directions: on the reference update the collision rate improved to zero while L2 agreement worsened, so no single system-level metric admits the update safely and the full terminal vector is required.

**Contributions:** This paper contributes the following.

- A dual-mode measurement method for retraining-induced cascades in modular MLS, with an isolation check.
- A controlled measurement, to our knowledge the first on a real perception–prediction–planning pipeline, of how single-component retraining under a geographic domain shift propagates downstream, run as a 15-update campaign that reveals strict entangled enhancement in 6 of 15 ordinary updates and a wide landscape of outcomes.
- A cascade-aware retraining assessment method (CARA) built on this measurement, whose interface-drift front-end and admission rule we evaluate on the campaign.

The remainder of the paper is organized as follows. Section II formalizes the system model, including the definition of entangled enhancement, and the dual-mode methodology. Section III presents the proposed CARA method. Section IV details the experimental setup and the retraining campaign. Section V reports the evaluation

results, both the empirical entanglement findings and the evaluation of CARA. Section VI discusses the mechanism and its implications, Section VII reviews related work, and Section VIII concludes.

## II. SYSTEM MODEL AND METHODOLOGY

### A. Formal Pipeline Model

We model the driving stack as a composition of three functions,

$$S = C_3 \circ C_2 \circ C_1, \quad S(x) = C_3(C_2(C_1(x))), \quad (1)$$

where  $x$  is a camera keyframe and the three components carry out the canonical perception–prediction–planning decomposition:

- **$C_1$  (Perception):**  $z_1 = C_1(x)$  is a set of detected objects, each with a class, a position, and an extent in the ego frame.
- **$C_2$  (Prediction):**  $z_2 = C_2(z_1)$  is, for every detected object, a forecast trajectory over the next several seconds.
- **$C_3$  (Planning):**  $z_3 = C_3(z_2)$  is the ego vehicle’s planned trajectory, a sequence of positions over a 3 s horizon.

The intermediate signals  $z_1$  and  $z_2$  are the pipeline interfaces, and each is a cascade channel: a change in  $C_1$ ’s output distribution shifts  $z_1$ , which its direct consumer  $C_2$  then propagates into  $z_2$  and  $C_3$ . Fig. 1 shows the pipeline annotated with the before/after numbers from the reference retraining of this study. Only  $C_1$  is retrained (Boston  $\rightarrow$  Singapore); the isolated (*iso*) metrics of the frozen  $C_2$  and  $C_3$  stay constant, while their pipeline (*pipe*) metrics move because their inputs shift.

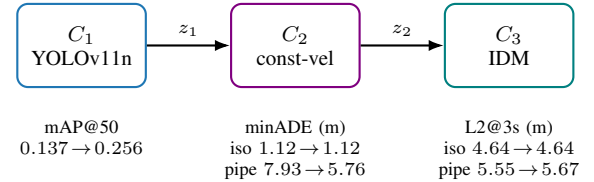


Fig. 1: Pipeline  $C_1 \rightarrow C_2 \rightarrow C_3$  with before/after numbers.

### B. Entangled Enhancement

The failure mode this paper investigates is *entangled enhancement*, named by the stochastic MLS model of Wang and Machida [3]. In words, an upstream component is retrained and improves on its own metric, yet the system as a whole degrades, because downstream components were tuned to the statistics of the *previous* upstream output and are disturbed when that output changes. Writing  $\delta_1$  for the change in the  $C_1$  metric, with positive  $\delta_1$  denoting an improvement, and  $\Delta_3$  for the change in the terminal (planning) pipeline metric, with positive  $\Delta_3$  denoting a regression, the strict condition is

$$\underbrace{\delta_1 > 0}_{C_1 \text{ improves}} \wedge \underbrace{\Delta_3 > 0}_{\text{system degrades}}. \quad (2)$$

The condition is strict because it requires the upstream component to improve on its *own* metric ( $\delta_1 > 0$ ) while the system nonetheless degrades ( $\Delta_3 > 0$ ); a component that simply got worse and dragged the system down would be an ordinary regression, not entangled enhancement. As Section V reports, retraining C1 on full-trainval Singapore data satisfies this condition directly:  $\delta_1 > 0$  in every update, and in 6 of 15 the planner degrades at the same time.

### C. Dual-Mode Evaluation

The core instrument is the measurement of each downstream component in two modes. Let  $z_k^*$  denote the ground-truth value of interface  $k$ . In *isolated* mode a component receives ground-truth inputs, so for  $C_2$  and  $C_3$  we evaluate

$$z_2^{\text{iso}} = C_2(z_1^*), \quad z_3^{\text{iso}} = C_3(z_2^*). \quad (3)$$

In *pipeline* mode a component receives the live output of its upstream neighbor,

$$z_2^{\text{pipe}} = C_2(C_1(x)), \quad z_3^{\text{pipe}} = C_3(C_2(C_1(x))). \quad (4)$$

Isolated mode exposes a component's own error, and pipeline mode exposes its own error plus the error inherited from upstream. For a component  $C_k$  with error metric  $M_k$  (lower is better), the difference between the two is the inherited part, which we call the *cascade gap* at  $C_k$ ,

$$\Gamma_k = M_k^{\text{pipeline}} - M_k^{\text{isolated}}. \quad (5)$$

A retraining event  $R$  applied to an upstream component changes the pipeline metrics. If the system is properly modular, it leaves all isolated metrics of untouched components unchanged. We call this the *isolation property*:

$$M_k^{\text{isolated}}(\text{before } R) = M_k^{\text{isolated}}(\text{after } R) \quad \forall \text{ frozen } C_k. \quad (6)$$

Equation (6) is not assumed but treated as a testable property of the apparatus: any leakage of the retraining event into a frozen component's isolated metric indicates contamination, and we verify the equation bit-exactly in Section V. The cascade *injected* by  $R$  at component  $C_k$  is then

$$\Delta_k = M_k^{\text{pipeline}}(\text{after } R) - M_k^{\text{pipeline}}(\text{before } R), \quad (7)$$

which, given (6), equals the change in the cascade gap  $\Gamma_k$ .

### D. Cascade Diagnostic and Coupling Factor

The canonical cascade hypothesis is a conjunction of three conditions: the retrained upstream component improves on its own metric, the frozen terminal component is stable in isolation, and it worsens in the pipeline. Using  $\delta_1$  and  $\Delta_3$  from Section II-B and (7), we encode the diagnostic as

$$\text{CASCADE} \equiv (\delta_1 > 0) \wedge (\Delta_3^{\text{iso}} = 0) \wedge (\Delta_3 > \epsilon), \quad (8)$$

with  $\epsilon$  a small noise floor; we set  $\epsilon = 0$  throughout this study, so the terminal clause reduces to  $\Delta_3 > 0$ . When (8) holds, the coupling from the retrained component to the system output is summarized by the *cascade coupling factor*

$$\rho_{1 \rightarrow 3} = \frac{\Delta_3}{\delta_1}. \quad (9)$$

On a synthetic pipeline with a known planted cascade, (8)–(9) recover  $\rho_{1 \rightarrow 3} \approx 0.84$ , which serves as an apparatus check prior to the empirical runs. As Section V shows, the empirical campaign satisfies the conjunction (8) in its canonical form in 6 of 15 updates, where  $\delta_1 > 0$  while  $\Delta_3 > 0$ , so  $\rho_{1 \rightarrow 3}$  is well-defined and positive for those cases and is read together with the full cascade vector.

### E. Single-Component Retraining Under Domain Shift

Each retraining event follows the same procedure, and only C1 is ever changed:

- 1) Train C1 on the source domain (Boston scenes). Freeze C2 and C3 permanently.
- 2) Measure the five-quantity profile on the target-domain validation set (Singapore): C1 detection quality (mAP@50), C2-isolated minADE, C2-pipeline minADE, C3-isolated L2@{1, 2, 3}s, C3-pipeline L2@{1, 2, 3}s.
- 3) Fine-tune C1 on the target domain (Singapore scenes), producing the retrained C1'.
- 4) Re-measure the identical five-quantity profile with C1' in place.
- 5) Verify the isolation property (6) and compute the injected cascades (7) at C2 and C3.

The Boston-to-Singapore shift within nuScenes is a genuine and well-studied domain gap: left- versus right-hand traffic, different vehicle and agent mixes, and different lighting conditions [5].

### F. Metrics

C1 is scored with standard detection metrics (mAP@50, mAP@50-95, precision, recall) on the target-domain validation images. For  $C_2$ , the metric is the minimum average displacement error (minADE). For an agent with forecast positions  $\hat{p}_t^{(m)}$  under mode  $m$  and ground-truth positions  $p_t^*$  over  $T$  future steps,

$$\text{minADE} = \min_m \frac{1}{T} \sum_{t=1}^T \|\hat{p}_t^{(m)} - p_t^*\|_2, \quad (10)$$

averaged over agents. Our deterministic predictor has a single mode, so the min collapses to that mode. For  $C_3$ , the metric is the L2 displacement error between the planned ego trajectory  $\hat{e}_t$  and the human driver's recorded trajectory  $e_t^*$  at horizon  $\tau$ ,

$$\text{L2@}\tau = \|\hat{e}_\tau - e_\tau^*\|_2, \quad \tau \in \{1, 2, 3\} \text{ s}, \quad (11)$$

reported together with collision rate. We also report, for the campaign, a *plan shift*: the distance between the plan produced with the incumbent C1 and the plan produced with the retrained C1 on the same scene, which measures how far a retraining moves the driving decision independent of the human path. The L2 metric is the standard nuScenes open-loop planning protocol used by the E2E literature [2], [9], [10], with the differential-use caveat discussed in Sections V and VII.

### III. CASCADE-AWARE RETRAINING ASSESSMENT

The measurement method of Section II exposes a concrete hazard for maintenance: a retrained component that improves on its own metric, and even improves its neighbor, can still degrade the system, so neither component-level nor first-interface validation is a sufficient admission criterion. We turn the method into an admission check for component updates, Cascade-Aware Retraining Assessment (CARA). CARA formalizes, as an explicit gate, the procedure we later run across the campaign of Section IV. Fig. 2 summarizes the decision and Algorithm 1 states it. The evaluation of CARA on real updates is deferred to Section V.

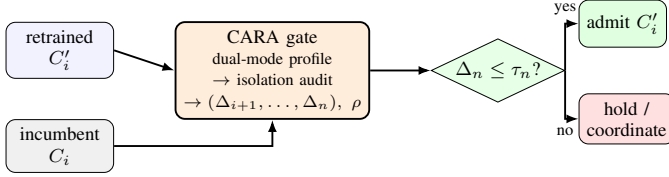


Fig. 2: The CARA admission check for a retrained component.

#### A. The Admission Procedure

Given a pipeline  $C_1 \rightarrow \dots \rightarrow C_n$ , a frozen audit set  $A$ , and a candidate update  $R$  to component  $C_i$ , CARA records the dual-mode profile with the incumbent  $C_i$ , swaps in the retrained  $C'_i$ , and records it again on the identical audit set with fixed randomness. It requires the isolation property (6) to hold for every frozen  $C_k$ . Any violation signals leakage or nondeterminism and invalidates the assessment. In our campaign this audit passed bit-exactly (F2). It then computes the injected cascade  $\Delta_k$  (7) at every downstream component and reports the full vector  $(\Delta_{i+1}, \dots, \Delta_n)$ , because adjacent deltas can carry opposite signs, together with the coupling factor  $\rho_{i \rightarrow n} = \Delta_n / \delta_i$ . Under the sign convention of Section II-B (a positive  $\delta_i$  improves the component, a positive  $\Delta_n$  worsens the system), a positive  $\rho$  with an improving  $\delta_i$  is the entangled-enhancement regime. As our campaign shows,  $\rho$  is well-defined only when  $\delta_i$  is unambiguous, which holds throughout the full-trainval campaign of Table II, where  $\delta_1 > 0$  in every update. For the six entangled-enhancement cases  $\rho_{1 \rightarrow 3} > 0$ , quantifying how strongly an improvement in C1 propagates into a planner regression. The update is admitted only if  $\Delta_n \leq \tau_n$  and safety-critical auxiliaries such as collision rate do not regress. Otherwise it is held or routed to coordinated retraining of the downstream consumers.

#### B. Interface-Drift Front-End

The full admission procedure runs the entire pipeline on the audit set, which is the cost of a complete evaluation. We precede it with a lower-cost screen that reads only the detector's own outputs. For a candidate detector, the screen compares the statistics of its detections against those of the incumbent on the same audit scenes, using a Kolmogorov–Smirnov statistic over the continuous features (per-image count, confidence, box-center position)

---

#### Algorithm 1: Cascade-Aware Retraining Assessment (CARA)

---

**Input:** pipeline  $C_1 \rightarrow \dots \rightarrow C_n$ ; audit set  $A$ ; candidate update  $R$  on  $C_i$ ; tolerance  $\tau_n$

**Output:** admit / hold decision, cascade vector, coupling factor  $\rho$

$P_{\text{before}} \leftarrow \text{DUALMODEPROFILE}(C_i, A)$   
 $C'_i \leftarrow R(C_i)$ ;  $P_{\text{after}} \leftarrow \text{DUALMODEPROFILE}(C'_i, A)$

**foreach** frozen  $C_k, k > i$  **do**  
   **if**  $M_k^{\text{iso}}(\text{after}) \neq M_k^{\text{iso}}(\text{before})$  **then**  
     **return invalid** // isolation violated (contamination)  
**end**

**end**

$\Delta_k \leftarrow M_k^{\text{pipe}}(\text{after}) - M_k^{\text{pipe}}(\text{before})$  for  $k = i+1..n$   
 $\rho \leftarrow \Delta_n / \delta_i$

**if**  $\Delta_n \leq \tau_n$  **and** collision rate not worse **then**  
   **return admit**  $C'_i$   
**else**  
   **return hold** / coordinate downstream retraining  
**end**

---

and a Jensen–Shannon divergence over the class histogram, aggregated into one interface-drift score in  $[0, 1]$ . This follows the between-stage two-sample tests used in production data-validation systems [11], and it is model-free: it consumes only the detection records the pipeline already emits. The screen identifies which updates warrant the full dual-mode assessment, and its predictive value is evaluated in Section V-H.

#### C. Why the Method Needs a Modular Architecture

Admission never rests on  $\delta_i$ , which is essential precisely because entangled enhancement (F4) is the case where  $\delta_i$  improves while the system regresses; admission instead reports the full delta vector, so an improvement at one interface cannot mask a regression further downstream. The isolation audit makes silent contamination detectable by exact equality testing, a check available only because the architecture is modular (F1). The whole method is therefore constructible only in the modular setting. In an E2E monolith there are no frozen components, isolated modes, or interfaces at which  $\Delta_k$  could be measured. Its cost is inference on a fixed audit set, with no training.

### IV. EXPERIMENTAL SETUP

#### A. Dataset and Components

We use the full nuScenes v1.0 trainval split [5]. A nuScenes *scene* is a  $\approx 20$  s driving clip, annotated at 2 Hz; each of these annotated samples is a *keyframe*, so a scene contributes about 40 keyframes, and we use the front-camera (CAM\_FRONT) keyframe as one evaluation frame throughout. The trainval split contains 850 scenes (34,149 keyframes in total,  $\approx 45$  GB), of which 467 scenes were recorded

in Boston and 383 in Singapore. Partitioning by recorded location and splitting each city 60/40 into train and validation gives `boston_train/boston_val` (280/187 scenes) and `singapore_train/singapore_val` (229/154 scenes). The Boston detector is trained on the 11,087 `CAM_FRONT` keyframes of `boston_train`. To keep evaluation affordable across the many fine-tunes of the campaign while spanning the whole city, all cascade measurements use a fixed subset of `singapore_val` obtained by an even stride over its 154 scenes down to 50 scenes, comprising  $n = 2,020$  annotated keyframes; this same 50-scene audit set is reused, unchanged, for every one of the 15 updates so the comparisons are exact. For C1 training, the 3D box annotations are projected onto the front-camera keyframes to produce 2D labels over eight driving classes.

C1 is YOLOv11n [6], a camera detector rather than a LiDAR one such as CenterPoint [12], so absolute magnitudes will differ with a richer perception front-end even though the cascade quantities are defined independently of it. It is fine-tuned for 10 epochs on `boston_train` from COCO weights to give the incumbent detector, then fine-tuned further on `singapore_train` starting from the Boston checkpoint to give each retrained detector. At inference, each 2D detection is lifted to an ego-frame ground position by intersecting its bottom-center pixel ray with the ground plane, supplying the agent positions  $z_1$  that C2 consumes. C2 is a deterministic, parameter-free constant-velocity predictor: in isolated mode it rolls agents out at their ground-truth velocity, and in pipeline mode it anchors a zero-velocity rollout at C1’s single-frame detections, so that any change in its pipeline output is attributable solely to a change in C1. C3 is a rule-based Intelligent Driver Model [7] proposal-and-score planner that selects, from a speed-aware grid of candidate trajectories  $\mathcal{P}$ , the one minimizing progress cost plus a soft proximity penalty against the predicted agent futures  $z_2$ ,

$$\pi^* = \arg \min_{\pi \in \mathcal{P}} J_{\text{prog}}(\pi) + \lambda \sum_{a \in z_2} \phi(d(\pi, a)), \quad (12)$$

where  $d(\pi, a)$  is the closest approach between candidate  $\pi$  and agent  $a$ ,  $\phi$  is a soft penalty on proximity, and  $\lambda$  weights safety against progress. This reproduces the PDM-Closed [8] architecture without its learned scorer; the dependence of (12) on  $z_2$  is the channel through which perturbed detections reach the plan.

### B. The Retraining Campaign

A single fine-tuning event is one point in a large space of possible updates. To map the space rather than sample it once, we run a campaign of 15 **retraining updates** of C1, each measured through the identical dual-mode harness of Section II-E. The campaign varies three knobs:

- **Fine-tuning data size:** the full `singapore_train` set (the 229-scene Singapore split), a deterministic half of it, and a quarter. Smaller sets are the harder, more data-starved cases.

- **Training length:** 5, 10, and 20 epochs, to test whether longer fine-tuning changes the downstream effect.
- **Random seed:** each setting is repeated with three seeds, so a result can be told apart from seed noise.

The matrix is three data sizes at 10 epochs and two extra training lengths (5 and 20) at the full data size, each over three seeds, giving  $3 \times 3 + 2 \times 3 = 15$  updates. The incumbent C1 (Boston-trained) and the frozen C2 and C3 are shared across all 15 updates, and only the Singapore fine-tune of C1 changes between them. Every randomness source is fixed within each update so that the isolation property (6) can be checked for exact equality rather than statistical closeness.

### C. Measurement Protocol

The five-quantity profile records one C1 metric together with the isolated and pipeline metrics of C2 and C3, so that each component’s own error and its inherited error are separated (Section II-E). For each of the 15 updates the harness records this profile on `singapore_val` for both the incumbent and the retrained C1, then computes  $\delta_1$ ,  $\Delta_2$ ,  $\Delta_3$ , the plan shift, and the regime label (Section II-B). Because C2 and C3 have no trainable parameters, the isolation check is exact.

## V. EVALUATION RESULTS

This section reports two evaluations. The first is the empirical study of the cascade itself: one update in full detail as a reference case (Sections V-B–V-E), then the campaign that places it in a landscape (Section V-F). The second evaluates the proposed CARA method on those same updates, through its interface-drift front-end as a predictor of downstream risk (Section V-H) and its admission rule against the measured ground truth (Section V-I).

### A. Reference Update: Headline Measurements

The reference update is the full-data, 10-epoch, seed-1 fine-tune. Table I reports its before/after profile on `singapore_val` ( $n = 2,020$  frames), where “before” is the Boston-trained  $C_1$  and “after” is the Singapore-fine-tuned  $C_1$ , and Fig. 3 plots it. The detection quality of  $C_1$  (mAP@50) *rises* after fine-tuning ( $0.137 \rightarrow 0.256$ ), C2-pipeline minADE improves ( $7.93 \rightarrow 5.76$  m), and yet C3-pipeline L2@3s still *worsens* ( $5.55 \rightarrow 5.67$  m). Both isolated metrics are unchanged, which is the visual signature of the isolation property (6). This single update is already an instance of entangled enhancement: the detector improves on its own metric while the system it feeds degrades.

### B. Verification of the Isolation Property (F2)

Before reading any cascade delta, the frozen components must be shown to be untouched, which is a validity check on the apparatus and not itself a result. The three isolated-mode rows are exactly zero-delta. C2-isolated minADE is 1.1211 m in both runs, and C3-isolated L2 is 0.7458/2.4100/4.6384 m at the three horizons in both runs, to every recorded digit. For deterministic, parameter-free C2 and C3 fed identical ground-truth inputs, this invariance is expected by construction, and

TABLE I: Reference update: measurements on Singapore validation ( $n = 2,020$  frames, full trainval).

| Metric                     | Before        | After         | $\Delta$       | $\Delta\%$   |
|----------------------------|---------------|---------------|----------------|--------------|
| <b>C1 mAP@50</b>           | <b>0.1370</b> | <b>0.2556</b> | <b>+0.1186</b> | <b>+86.6</b> |
| C2 iso. minADE (m)         | 1.1211        | 1.1211        | 0.0000         | 0.00         |
| <b>C2 pipe. minADE (m)</b> | <b>7.9310</b> | <b>5.7590</b> | <b>-2.1720</b> | <b>-27.4</b> |
| C3 iso. L2@3s (m)          | 4.6384        | 4.6384        | 0.0000         | 0.00         |
| <b>C3 pipe. L2@3s (m)</b>  | <b>5.5463</b> | <b>5.6703</b> | <b>+0.1240</b> | <b>+2.2</b>  |
| C3 pipe. collision rate    | 0.020         | 0.000         | -0.0200        | n/a          |

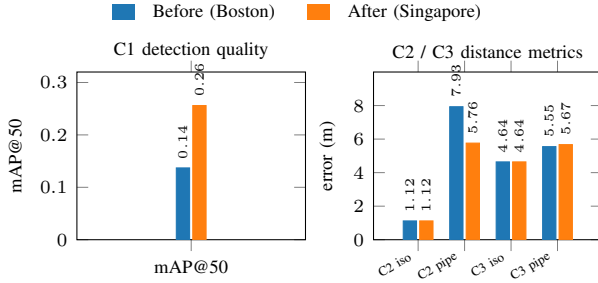


Fig. 3: Reference update on `singapore_val`, before versus after: (a) C1 detection quality, (b) C2/C3 distance metrics.

any nonzero delta would have signalled state or data leakage between the runs. Its role is to certify that the pipeline-mode deltas in Table I arise solely from the C1 retraining event, transmitted through the component interfaces. This exact-equality check held across all 15 updates in the campaign.

#### C. Cascade Concentration at the C1→C2 Interface (F3)

The largest single effect in Table I occurs at the C1→C2 interface. With the Boston-trained C1, C2’s pipeline-mode minADE was 7.9310 m against an isolated-mode floor of 1.1211 m, a cascade gap  $\Gamma_2$  of 6.81 m. This  $\Gamma_2$  conflates two sources: C1 detection error and the loss of velocity information, since C2-isolated uses ground-truth velocity whereas C2-pipeline uses a zero-velocity rollout (Section IV); it therefore bounds the combined effect, not perception error alone. After the Singapore fine-tune, C2-pipeline minADE fell to 5.7590 m ( $\Gamma_2 = 4.64$  m), a 27.39% improvement without modifying C2. The change  $\Delta_2 = -2.17$  m is attributable to C1 alone, since the velocity-information gap is constant across the two runs, so the first downstream interface dominates the retraining-induced cascade.

#### D. Strict Entangled Enhancement at the System Output (F4)

The reference update satisfies every clause of the diagnostic (8) at once. The detector improves on its own metric ( $\delta_1 = +0.119$ , mAP@50  $0.137 \rightarrow 0.256$ ), its interface to C2 improves ( $\Delta_2 = -2.17$  m, a 27% reduction), the isolation clause holds ( $\Delta_3^{\text{iso}} = 0$ ), and yet the planner’s trajectory-agreement metric degrades ( $\Delta_3 = +0.124$  m  $> \epsilon$ ;  $5.55 \rightarrow 5.67$  m). This is the strict entangled-enhancement condition of the base paper [3]: an upstream component better on its own metric and on its neighbor’s still moves the terminal

metric it feeds. The effect is best read as a *divergence* of the terminal metrics rather than an unqualified harm: on this same update the pipeline collision rate *improved*, from 2% under the incumbent to zero (Table I), even as L2 agreement worsened. A better detector thus pushes the two terminal readings in opposite directions, which is exactly why a single-metric admission gate on the system output is unsafe and the full delta vector (Section III) is needed. It is a controlled, quantified instance of the correction-cascade pattern that the technical-debt literature describes only qualitatively [13]. Fig. 4 illustrates it: C3-isolated remains fixed at 4.64 m, since the planner receives identical ground-truth inputs in both runs, while C3-pipeline drifts from 5.55 to 5.67 m.

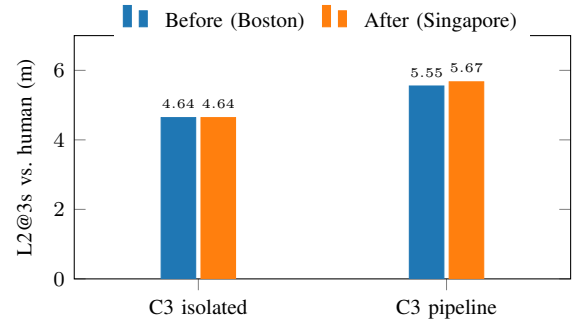


Fig. 4: Cascade signature on planning: isolated fixed, pipeline drifts.

#### E. The Detector Improves, Yet the System Degrades

The reference update shows that the entangled enhancement is not driven by a degraded detector: C1’s aggregate metric *rises* by 86.6% (mAP@50  $0.137 \rightarrow 0.256$ ). The 229-scene Singapore fine-tune set is large enough that adaptation to the new city genuinely improves the detector rather than over-fitting it. The confusion matrix of the Boston-trained detector (Fig. 5) shows the dominant `car` class well separated, with the rarer classes still frequently confused with background, a residual class imbalance that motivates the class-balanced study of Section V-G. Crucially, the improvement in C1’s own score means the cascade cannot be attributed to a weaker perception front-end: it is a genuine instance of a better component harming the system.

#### F. The Campaign: A Landscape, Not a Constant (F5)

The reference update is a single point in this space. Table II reports all 15 updates of the campaign (Section IV-B), and three patterns emerge.

First, the detector improves on its own metric in *every* update:  $\delta_1 > 0$  in all 15 (Table II), and prediction improves too ( $\Delta_2 < 0$  in all 15). Fine-tuning on the new city genuinely sharpens C1 across the whole campaign.

Second, and most importantly, this is enough to observe *entangled enhancement directly*: in 6 of the 15 updates  $\delta_1 > 0$  while  $\Delta_3 > 0$ , i.e. the detector improves yet the planner degrades. The six span a range of severities, from a marginal

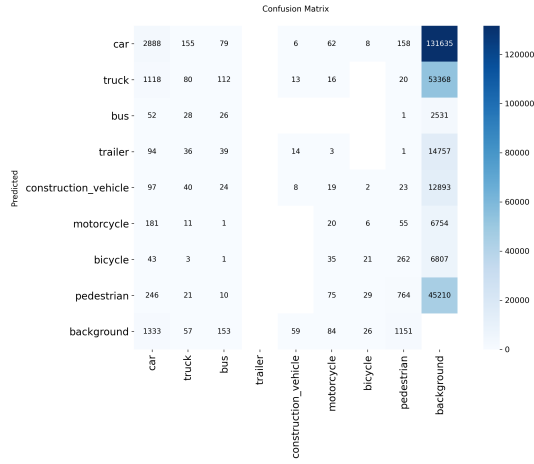


Fig. 5: Confusion matrix of the Boston-trained  $C_1$  on full trainval.

+0.008 m up to a material +0.193 m. No class re-balancing is required to reach this condition; it arises in the ordinary campaign.

Third, the direction of the terminal effect is campaign-dependent: the planner is worse ( $\Delta_3 > 0$ ) in 6 updates and better ( $\Delta_3 < 0$ ) in the other 9. The plan shift ranges from 0.78 m to 1.50 m over the 2,020-frame evaluation set. Which updates land in the entangled-enhancement regime is not fully predictable from the training configuration, which is precisely why a downstream check (Section III) is needed rather than a rule on the detector alone.

TABLE II: The 15-update retraining campaign on singapore\_val ( $n = 2,020$  frames).  $\delta_1$ : detector-score change (positive better);  $\Delta_2$ : prediction-error change (negative better);  $\Delta_3$ : planner-error change (positive = worse); shift: plan shift in metres. With noise floor  $\epsilon = 0$ , “ent. enh.” marks entangled enhancement ( $\delta_1 > 0$  with  $\Delta_3 > 0$ ) and “helped” marks  $\Delta_3 < 0$ ; every update has  $\delta_1 > 0$ .

| Update             | data | $\delta_1$ | $\Delta_2$ | $\Delta_3$ | shift | regime    |
|--------------------|------|------------|------------|------------|-------|-----------|
| Full, 10 ep, s1    | full | +0.119     | -2.17      | +0.124     | 1.05  | ent. enh. |
| Full, 10 ep, s2    | full | +0.122     | -2.28      | +0.193     | 1.02  | ent. enh. |
| Full, 10 ep, s3    | full | +0.129     | -2.15      | +0.187     | 0.92  | ent. enh. |
| Half, 10 ep, s1    | half | +0.114     | -2.55      | -0.121     | 1.35  | helped    |
| Half, 10 ep, s2    | half | +0.120     | -2.71      | -0.230     | 1.19  | helped    |
| Half, 10 ep, s3    | half | +0.106     | -1.77      | -0.151     | 1.46  | helped    |
| Quarter, 10 ep, s1 | qtr  | +0.121     | -2.71      | +0.073     | 0.79  | ent. enh. |
| Quarter, 10 ep, s2 | qtr  | +0.113     | -1.29      | -0.093     | 0.78  | helped    |
| Quarter, 10 ep, s3 | qtr  | +0.101     | -2.82      | +0.008     | 1.00  | ent. enh. |
| Full, 5 ep, s1     | full | +0.138     | -1.08      | -0.095     | 0.89  | helped    |
| Full, 5 ep, s2     | full | +0.122     | -1.88      | -0.130     | 1.22  | helped    |
| Full, 5 ep, s3     | full | +0.130     | -1.08      | -0.041     | 0.92  | helped    |
| Full, 20 ep, s1    | full | +0.142     | -2.59      | +0.037     | 1.37  | ent. enh. |
| Full, 20 ep, s2    | full | +0.126     | -1.77      | -0.083     | 1.50  | helped    |
| Full, 20 ep, s3    | full | +0.130     | -2.44      | -0.020     | 1.20  | helped    |

## G. Robustness to Class Imbalance

The full-trainval campaign already yields strict entangled enhancement without any intervention on the training data (F4). A natural question is whether the effect is an artifact of the residual class imbalance visible in Fig. 5, where rare classes are still confused with background. To test this, we repeat the campaign with a class-balanced training split: images containing rare classes are oversampled so the per-class instance distribution flattens, with a repeat factor  $(n_{\max}/n_c)^p$  per class  $c$  clipped to  $[1, 12]$ , while the validation split is left untouched so metrics stay comparable. We run two balance strengths ( $p \in \{0.5, 1.0\}$ ), two training lengths (10 and 20 epochs), and three seeds, giving 12 updates, of which 7 completed within the compute budget (6 at  $p=0.5$  and 1 at  $p=1.0$ ).

The effect persists under class balancing. Across the 7 completed updates the detector still improves in every case ( $\delta_1 > 0$  in 7/7), and strict entangled enhancement appears in 4 of the 7: three at  $p=0.5$  (ep10\_s3, ep20\_s1, ep20\_s2, with  $\delta_1$  up to +0.143 and  $\Delta_3$  up to +0.139 m) and one at  $p=1.0$  (ep20\_s1,  $\delta_1=+0.125$ ,  $\Delta_3=+0.075$  m). Both balance strengths and both training lengths that reached the entangled-enhancement regime did so at 20 epochs, matching the pattern seen without balancing. Rebalancing the rare classes therefore does not remove the effect; the strict condition is not an artifact of the residual class imbalance but a property of the pipeline under domain-shift retraining.

## H. Evaluation of CARA’s Drift Screen

The campaign enables an evaluation of the interface-drift front-end of CARA (Section III-B) as a predictor of risk. For each of the 15 updates we compute the drift score of the retrained detector against the incumbent on the audit scenes and assess how well it anticipates the downstream disruption that only the full pipeline reveals.

Ranked against the plan shift each update caused, the drift score is a moderate predictor of *how much* the plan moves: the rank correlation between drift and plan shift is  $\approx 0.56$  (Fig. 6). Separating *L2-regressing* updates ( $\Delta_3 > 0$ ) from non-regressing ones is a harder task, however, and the drift score does not do it well: its AUROC against the L2-regression label is  $\approx 0.48$ , essentially chance. The reason is visible in Fig. 6: every retrained detector improved consistently, so all drift scores fall in a narrow band (0.05 to 0.08), and the regressing and non-regressing updates are interleaved within it. The front-end therefore carries usable signal about the *magnitude* of downstream disturbance but not its *direction*, so it is best used to prioritize updates for the full dual-mode assessment rather than as a standalone admit/hold rule. The cheap screen is a first filter, and the full admission procedure of Algorithm 1, which measures  $\Delta_n$  directly, remains the reliable decision.

## I. Evaluation of CARA’s Admission Rule

We next evaluate the drift score as the binary admit/hold rule of CARA’s front-end, scored against ground truth. We label an update “L2-regressing” when it worsened the planner’s

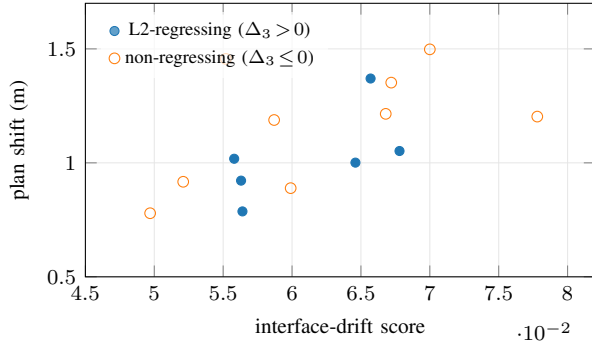


Fig. 6: Interface drift vs. plan shift over the 15 full-trainval updates (Spearman  $\approx 0.56$ ). Drift tracks the magnitude of plan movement, but L2-regressing and non-regressing updates are interleaved, so drift alone does not classify the L2 direction (AUROC  $\approx 0.48$ ).

trajectory-agreement metric ( $\Delta_3 > 0$ ); of the 15 full-trainval updates, 6 are L2-regressing and 9 are not. Treated as a ranking, the drift score places an L2-regressing update above a non-regressing one only about half the time (AUROC  $\approx 0.48$ ), consistent with the narrow, interleaved drift band of Fig. 6.

As a thresholded decision the rule is therefore weak, as Table III shows. At a strict threshold every update is held, so no L2-regressing update is admitted (recall 1.0) but no update is admitted at all (specificity 0), which is safe but provides no discrimination. The best balanced operating point still catches every L2-regressing update (recall 1.0) but admits only two of nine non-regressing ones (specificity 0.22), at precision 0.46: the front-end can be tuned to catch every L2 regression, but at the cost of holding most other updates too. The two terminal clauses of Algorithm 1 disagree here, which is the point. The incumbent pipeline had a 2% collision rate, and every retrained update drove it to zero, so the collision-rate clause would admit all 15; the terminal-L2 clause, in contrast, holds the six  $\Delta_3 > 0$  updates. CARA holds those six on L2 even though their collision rate improved, so neither clause alone is a safe gate: the value of the method is that it reports the full terminal vector and surfaces the disagreement for a human or a coordinated downstream retrain, rather than collapsing it into one admit/hold bit.

TABLE III: CARA drift-rule decisions against ground truth ( $\Delta_3 > 0 = \text{L2-regressing}$ ), on the 15 full-trainval updates (6 L2-regressing, 9 not). The held-out row tunes the threshold on a 7-update tune split and reports on the untouched 8-update test split.

| Threshold         | TP/FP/FN/TN | Rec. | Spec. | Prec. | F1   |
|-------------------|-------------|------|-------|-------|------|
| Strict (hold all) | 6/9/0/0     | 1.00 | 0.00  | 0.40  | 0.57 |
| Balanced          | 6/7/0/2     | 1.00 | 0.22  | 0.46  | 0.63 |
| Held-out (test)   | 3/3/0/2     | 1.00 | 0.40  | 0.50  | 0.67 |

We interpret these results conservatively. The threshold is validated on a held-out split: it is chosen on a 7-update

tune half and applied to the untouched 8-update test half, so the reported operating point is not fitted on the data it is scored against. On that held-out test the rule still catches every L2-regressing update (recall 1.0) but keeps a low specificity (0.40), confirming that the cheap drift proxy is a prioritization filter, not a decision rule. With only 8 test updates the estimates are wide: Wilson 95% intervals are [0.44, 1.00] recall, [0.12, 0.77] specificity, and [0.19, 0.81] precision, so they are indicative rather than tight. The full admission procedure of Algorithm 1, which runs the complete dual-mode profile rather than the drift proxy, remains the reliable decision, and the front-end serves as its low-cost first filter that never misses an L2-regressing update.

### J. Training Diagnostics

Both fine-tunes converged within their epoch budgets with monotone loss decay, as Fig. 7 shows for the Boston run: the box, classification, and distribution-focal losses decrease on both the train and validation splits, while precision, recall, and mAP@50 increase, with mAP@50 rising from 0.116 at epoch 1 to 0.137 at epoch 10. Each fine-tune completes in under a GPU-minute, several orders of magnitude below the cost of a single E2E training run [14], which is what renders a campaign of repeated retraining feasible. The cascade propagates through the ego-frame positions of the detected boxes, which alter the forecasts and hence the plan, independently of whether the detections differ visibly to the eye.

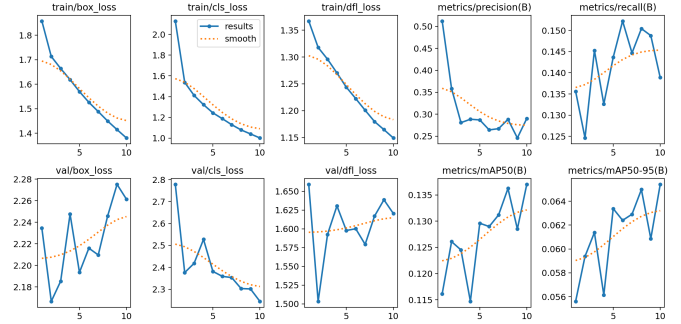


Fig. 7: YOLOv11n  $C_1$  Boston fine-tune training curves.

## VI. DISCUSSION

We explain the mechanism behind the non-monotone cascade and place CARA among related maintenance techniques.

### A. Why the Cascade Is Non-Monotone

Under the Boston-trained  $C_1$ , the planner’s inputs were substantially displaced ( $\Gamma_2 = 6.81$  m) in a regime whose errors the proposal-and-score rules absorbed: missed or grossly misplaced agents leave the proximity penalty inactive, so the planner follows its kinematic prior, which on these scenes approximated the human trajectory. The fine-tuned  $C_1$  relocated detections near their true positions and re-activated the penalty with correctly anchored but coarse (zero-velocity)

forecasts, causing the planner to react to obstacles it had previously ignored. Downstream components can thus absorb upstream error, and, symmetrically, upstream improvement can expose masked downstream sensitivity. A retrained component therefore cannot be admitted on its own interface alone.

This mechanism also cautions against a literal interpretation of the terminal metric. The collision rate in fact *fell*, from 2% under the incumbent to zero after retraining, even as L2 rose, and the higher L2 of the reference case arises because the improved detector caused the planner to react to real agents rather than follow a kinematic prior that happened to track the recorded human path. This is the ego-forecasting shortcut of nuScenes open-loop L2, under which perception-agnostic prediction is competitive [10] and open-loop error is weakly tied to closed-loop quality [15]. A regression on this metric may therefore mark a *safer* planner that scores worse on a measure rewarding ignored perception, so the terminal readings here should be read as reduced trajectory agreement, not an established safety regression, and a closed-loop or NAVSIM-style metric [16] is the appropriate safeguard.

### B. CARA Among Related Maintenance Techniques

The findings give three maintenance rules that CARA (Section III) enforces directly. An update must not be admitted on the retrained component’s own metric, because the entangled-enhancement cases of F4 are exactly those in which C1 improved while the system degraded. Admission must not stop at the first interface, because those same cases let an improvement at C2 coincide with a regression at C3. And because the terminal effect is a landscape (F5) rather than a constant, a maintenance policy should assess several candidate updates rather than trust one.

CARA is one of several complementary techniques for these rules, not a replacement for them. Regression-constrained retraining such as positive-congruent training [17] and backward-compatible training [18] *reduces* the cascade at its source by constraining the update. Uncertainty-propagating interfaces [19] *absorb* the residual by passing distributions instead of point estimates across the perception–prediction boundary. Between-stage data validation [11] *monitors* the interface in production. CARA adds the missing *admission* step that measures the downstream cascade directly before an update is deployed, and its low-cost interface-drift front-end (Section III-B) can operate alongside the same production monitors.

## VII. RELATED WORK

### A. End-to-End Driving and Its Maintenance Limits

UniAD put detection, tracking, mapping, motion forecasting, occupancy prediction, and planning into a single planning-oriented network [2]. VAD replaced dense rasterized scene representations with vectorized ones for efficiency [9]. SparseDrive introduced fully sparse scene representations and reported the resource profile of the flagship systems:  $7.2\times$  faster training and  $5\times$  faster inference than UniAD, whose training takes about 144 GPU-hours [14].

The main survey of the field lists interpretability, causal confusion, and robustness under distribution shift as open challenges, and it does not treat component-level maintenance or retraining policy at all [1].

A recent E2E repair literature addresses maintenance directly. CorrectAD builds a self-correction loop that diagnoses failures, generates targeted training video with a diffusion model, and fine-tunes the entire E2E planner on old plus new data, correcting 62.5% of failure cases on nuScenes [20]. R2SE avoids full-model retraining by freezing the generalist model and training reinforcement-learned low-rank adapter specialists on failure regions, motivated by the catastrophic forgetting that naive fine-tuning causes [21]. Both reintroduce modular structure to make maintenance tractable, motivating the modular architecture adopted in this work.

### B. Evaluation of Driving Systems

Our planning metric is open-loop displacement error on nuScenes, and the limits of this protocol are well documented. Codevilla et al. showed as early as 2018 that offline prediction error correlates only weakly with actual closed-loop driving quality [15]. Zhai et al. found that a trivial MLP using only ego state, with no perception at all, matches perception-based E2E methods on nuScenes open-loop L2 [10]. Dauner et al. showed that open-loop ego-forecasting and closed-loop driving are misaligned tasks, and that a largely rule-based planner, PDM-Closed, outperformed learned planners [8]. Their later NAVSIM benchmark provides short-horizon non-reactive metrics that align far better with closed-loop performance [16]. These critiques target open-loop metrics used to *rank* architectures; our use is differential (the same frozen planner, same scenes, one upstream change), so much of that weakness cancels, and we report collision rate alongside L2 and revisit the caveat in Section VI.

### C. Model Updates, Regression, and Interface Compatibility

What we measure at system scale has per-sample analogues in the model-update literature. Yan et al. showed that model updates cause negative flips, samples the old model handled correctly that the new model gets wrong, at rates of 4–5% even between equal-accuracy retrains of identical architectures, and they proposed positive-congruent training with focal distillation to suppress them [17]. The backward-compatible training of Shen et al. constrains a new upstream representation model so that unchanged downstream consumers keep working [18], which is the same situation as updating C1 beneath a frozen C2 and C3. On the monitoring side, the data validation system of Breck et al. runs two-sample distribution tests between pipeline stages in production [11]. On a nuScenes detector–tracker–predictor stack nearly the same shape as ours, Ivanovic et al. showed that propagating state uncertainty across the perception–prediction interface, instead of passing point estimates, improves forecasting by a wide margin [19]. These works provide mitigation techniques. This paper contributes the complementary controlled measurement of the cascade that such techniques are intended to mitigate.

#### D. Reliability Modeling of Machine Learning Systems

The technical-debt analysis of Sculley et al. introduced CACE, unstable data dependencies, and correction cascades as qualitative failure modes of ML systems [13]. The base paper for this work, Wang and Machida [3], formalized imperfect retraining of a two-component MLS as a continuous-time Markov chain, named entangled enhancement, and compared progressive and conservative maintenance policies. Their follow-on work casts the same problem as an availability–continuity trade-off [4]. Both are analytical models whose state space grows as  $2^N$  for  $N$  components, and both are calibrated on synthetic transition rates. This paper provides an empirical counterpart: the isolation property we verify (F2) is the property their state factorization assumes, and the coupling factor we propose (Section III) is a measured statistic that can replace those synthetic rates.

#### VIII. CONCLUSION

We presented a controlled study of single-component retraining in a modular perception–prediction–planning pipeline under a geographic domain shift. Using dual-mode evaluation with a verified isolation control on the full nuScenes trainval split (2,020 evaluation frames), and running a campaign of 15 retraining updates rather than a single event, we found that retraining C1 disturbs the downstream plan in every case. The retrained detector improves on its own metric in all 15 updates, and yet in 6 of them the planner’s trajectory-agreement metric worsens at the same time. This is the strict form of entangled enhancement, in which a component better on its own metric and on its neighbor’s still moves the system output, observed directly, with no data-balancing trick required. The terminal metrics moreover *diverge*: on the reference update the collision rate improved to zero while L2 agreement worsened, so a better detector pushes the two system-level readings in opposite directions. Entangled enhancement is therefore not only an analytically modeled hazard but an observable one in a real driving pipeline. Together these results show that component-level validation, and any single-metric check on the system output, is insufficient for safe MLS maintenance, since each of those six updates would have passed a review of C1 alone and a review of the collision rate alone. We proposed the CARA admission method for maintaining such pipelines and evaluated its interface-drift front-end on a held-out split. The front-end tracks the magnitude of downstream disturbance (Spearman  $\approx 0.56$ ) but does not classify harm on its own (AUROC  $\approx 0.48$ ), so it is a prioritization filter for the full dual-mode admission procedure rather than a standalone decision rule. The broader contribution is methodological: cascade effects can be measured under exact controls, but only in an architecture that exposes its interfaces.

#### REFERENCES

- [1] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, “End-to-end autonomous driving: Challenges and frontiers,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [2] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, L. Lu, X. Jia, Q. Liu, J. Dai, Y. Qiao, and H. Li, “Planning-oriented autonomous driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [3] Z. Wang and F. Machida, “Maintaining performance of a machine learning system against imperfect retraining,” in *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2024, pp. 1782–1787.
- [4] Z. Wang and F. Machida, “Exploiting the availability–continuity trade-off in imperfect retraining of machine learning systems,” in *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, 2025, pp. 406–417.
- [5] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, “nusenes: A multimodal dataset for autonomous driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11 621–11 631.
- [6] G. Jocher and J. Qiu, “Ultralytics yolo11,” <https://github.com/ultralytics/ultralytics>, 2024.
- [7] M. Treiber, A. Hennecke, and D. Helbing, “Congested traffic states in empirical observations and microscopic simulations,” *Physical Review E*, vol. 62, no. 2, pp. 1805–1824, 2000.
- [8] D. Dauner, M. Hallgarten, A. Geiger, and K. Chitta, “Parting with misconceptions about learning-based vehicle motion planning,” in *Conference on Robot Learning (CoRL)*, 2023.
- [9] B. Jiang, S. Chen, Q. Xu, B. Liao, J. Chen, H. Zhou, Q. Zhang, W. Liu, C. Huang, and X. Wang, “Vad: Vectorized scene representation for efficient autonomous driving,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [10] J.-T. Zhai, Z. Feng, J. Du, Y. Mao, J.-J. Liu, Z. Tan, Y. Zhang, X. Ye, and J. Wang, “Rethinking the open-loop evaluation of end-to-end autonomous driving in nusenes,” *arXiv preprint arXiv:2305.10430*, 2023.
- [11] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, “Data validation for machine learning,” in *Proceedings of Machine Learning and Systems (SysML/MLSys)*, vol. 1, 2019.
- [12] T. Yin, X. Zhou, and P. Krähenbühl, “Center-based 3d object detection and tracking,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 11 784–11 793.
- [13] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden technical debt in machine learning systems,” in *Advances in Neural Information Processing Systems (NIPS)*, vol. 28, 2015, pp. 2503–2511.
- [14] W. Sun, X. Lin, Y. Shi, C. Zhang, H. Wu, and S. Zheng, “Sparsedrive: End-to-end autonomous driving via sparse scene representation,” *arXiv preprint arXiv:2405.19620*, 2024.
- [15] F. Codevilla, A. M. López, V. Koltun, and A. Dosovitskiy, “On offline evaluation of vision-based driving models,” in *European Conference on Computer Vision (ECCV)*, 2018.
- [16] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitshenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta, “Navsim: Data-driven non-reactive autonomous vehicle simulation and benchmarking,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.
- [17] S. Yan, Y. Xiong, K. Kundu, S. Yang, S. Deng, M. Wang, W. Xia, and S. Soatto, “Positive-congruent training: Towards regression-free model updates,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 14 299–14 308.
- [18] Y. Shen, Y. Xiong, W. Xia, and S. Soatto, “Towards backward-compatible representation learning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 6368–6377.
- [19] B. Ivanovic, Y. Lin, S. Shrivastava, P. Chakravarty, and M. Pavone, “Propagating state uncertainty through trajectory forecasting,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022.
- [20] E. Ma, L. Zhou, T. Tang, J. Zhang, J. Jiang, Z. Zhang, D. Han, K. Zhan, X. Lang, H. Sun, X. Zhou, D. Lin, and K. Yu, “Correctad: A self-correcting agentic system to improve end-to-end planning in autonomous driving,” *arXiv preprint arXiv:2511.13297*, 2025.
- [21] H. Liu, T. Li, H. Yang, L. Chen, C. Wang, K. Guo, H. Tian, H. Li, H. Li, and C. Lv, “Reinforced refinement with self-aware expansion for end-to-end autonomous driving,” *arXiv preprint arXiv:2506.09800*, 2025.